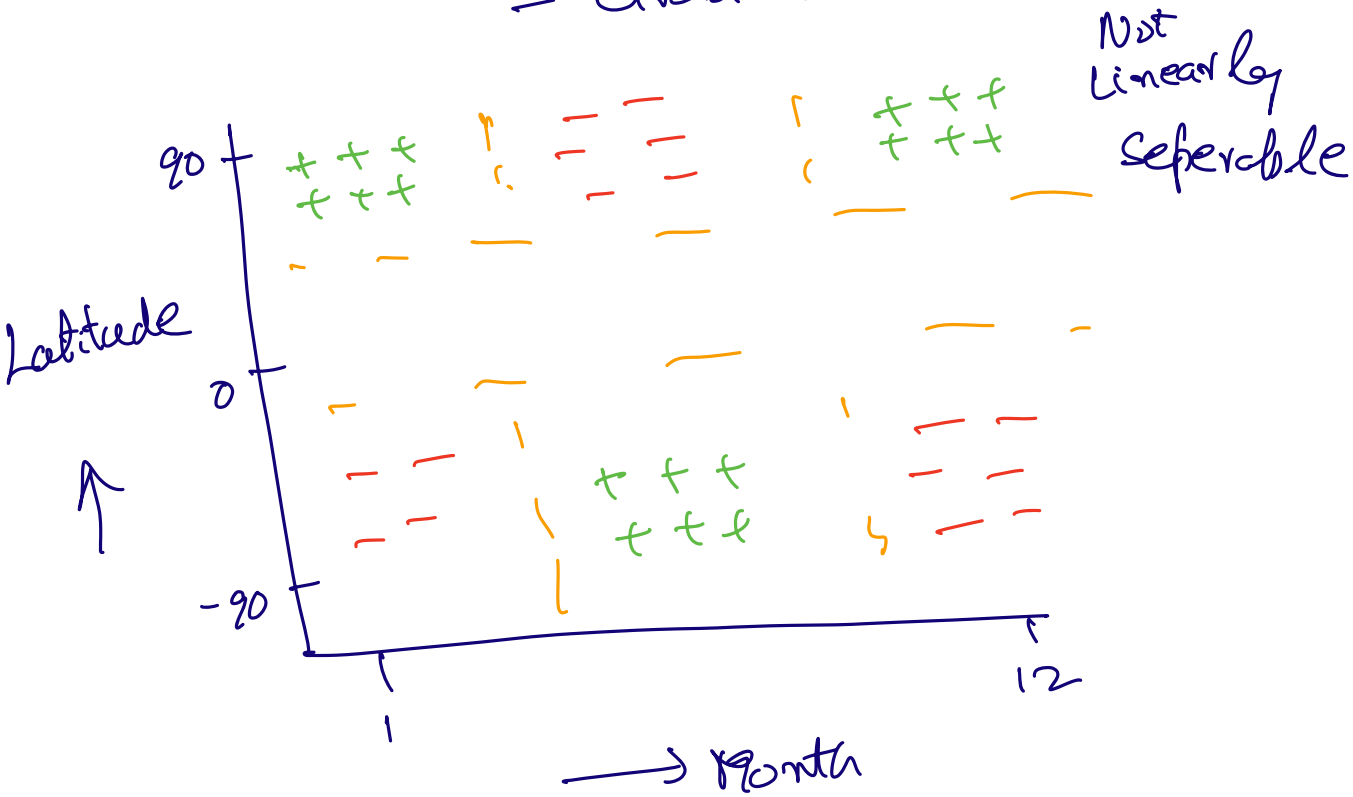
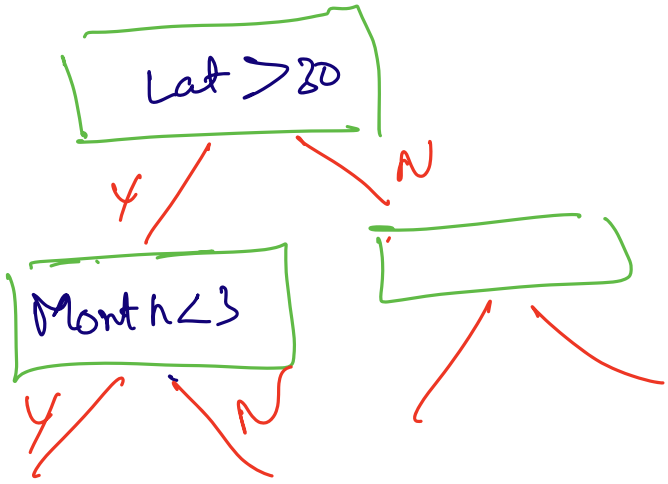
# Decision Trees

- Basics ✓
- Parameters ✓
- Regularisation ✓
- Ensembling ✓
  - Bagging ✓, Bootstrapping ✓
  - Boosting ✓
    - Ada ✓
    - Gradient ✓



- can we partition into different regions
- Choose a partition function which gives the best partition → greedy



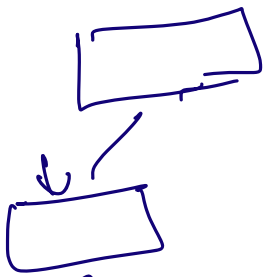

We need split function at each step of partition

$S_p(j, t)$ = $(\{X \mid X_j < t, X \in R_p\} \rightarrow R_1,\ \{X \mid X_j \geq t, X \in R_p\} \rightarrow R_2)$

$j$ ↑ feature number, $t$ ↑ threshold

Do partition ~~is done~~ s.t there

minimise misclassification : $1 - \max_c \hat{p}_c$

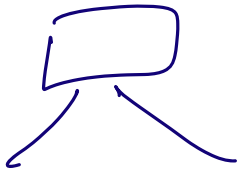


subset

$\hat{p}_c$ → how many belong to class $c$

Cross entropy

Gini Loss



$$\text{cross entropy} = \sum_{c} \hat{p}_c \log_2 \hat{p}_c$$

$$\text{Gini Loss or Gini Impurity}^2 \quad \sum_{c} \hat{p}_c (1 - \hat{p}_c)$$

## Decision Tree Regression

$$\text{Prediction } \hat{y}_m = \frac{\sum_{i \in R_m} y^{(i)}}{|R_m|}$$

$$\text{Loss in squared error} = \frac{\sum_{i \in R_m \text{ label}} (y^{(i)} - \hat{y}_m)^2}{|R_m|}$$

## D.T classification

$q$ categories $\rightarrow 2^q$ splits

D.T $\rightarrow$ high variance $\rightarrow$ overfit

## Regularisation

1. Minimum leaf size
2. Max depth of the tree
3. Max no: of nodes
4. Pruning
5. Max no: of features considered at each partition

Order: train $O(nfd)$ test $O(d)$

$n \rightarrow$ no: of samples

$f \rightarrow$ no: of features

$d \rightarrow$ depth of the tree

Scikit-learn CART (Classification and Regression Tree)

$$J(k, t_k) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$$

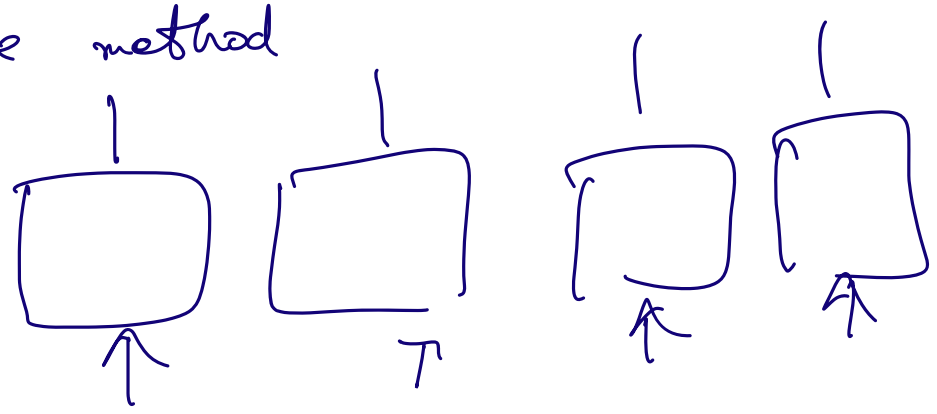
$k$: feature, $t_k$: threshold

$G_{left}$: Gini Impurity, $G_{right}$: Gini Impurity

---

## Ensembling of predictors

- Group of predictors
- Ensemble learning
- Ensemble method



Pros

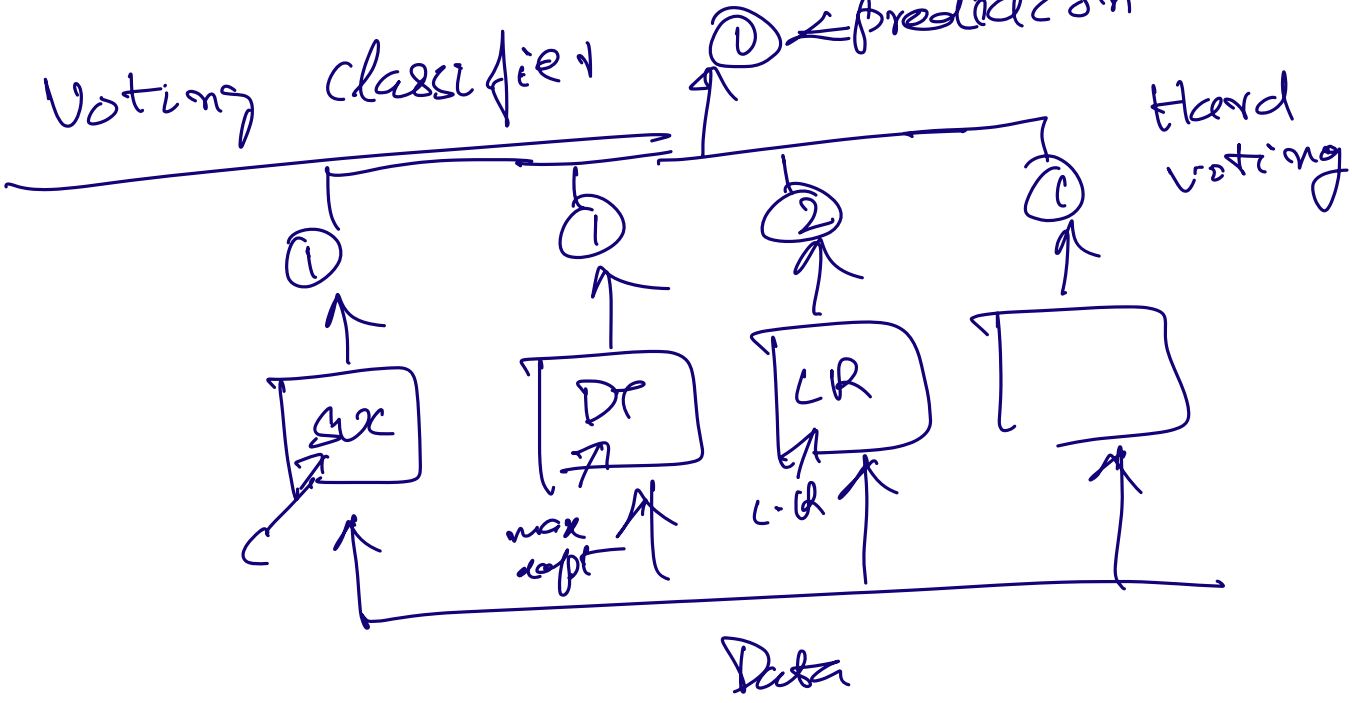
↳ Reduce variance

↳ Accuracy will better than individual predictions

Cons

↳ More compute

↳ Complexity deploy & mgmt

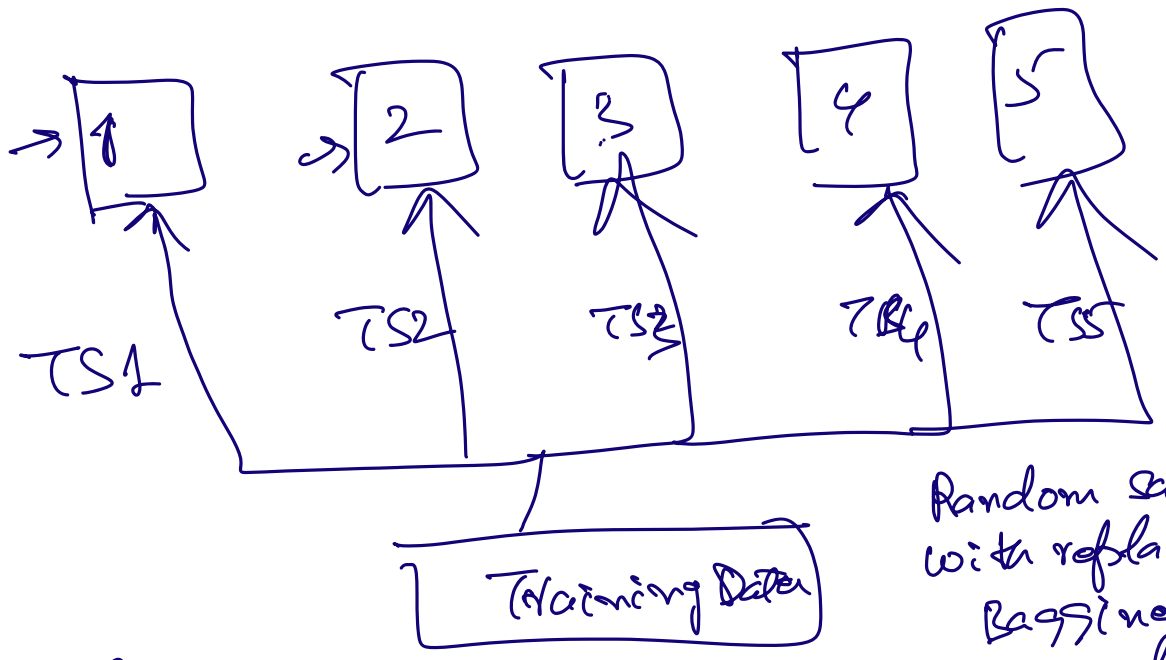



- Feeding each classifier different training data. Diff classifiers may focus different features
- SVC, Logistic Regression, DT → Diverse predictors
- Leads to higher accuracy

## Soft Voting

- Class probability from each predictor
- Average class probability
- Which class has → max avg probability

## Bagging & Pasting




Random sampling with replacement
Bagging

Random sampling w/o replacement
Pasting

- Lower Variance
- Good for predictors which has high variance (DecisionTree)

## Out of Bag Data

- 37% of samples may not get used
- OOB Score → to be used

## Random Patches & Random Subspaces

Random Patches → Random sampling of training data → Bagging → Pasting
→ Random pick of subset of features

Random Subspaces → only sampling features
Training data is ~~the~~ same

→ bias will increase
→ Variance will reduce

## Random Forest

- During classification it only considers $\sqrt{n}$ features, where $n$ → no. of features
- This brings diversity
- can increase bias but reduces variance

## Extra-Trees

↳ Extremely Randomised Trees

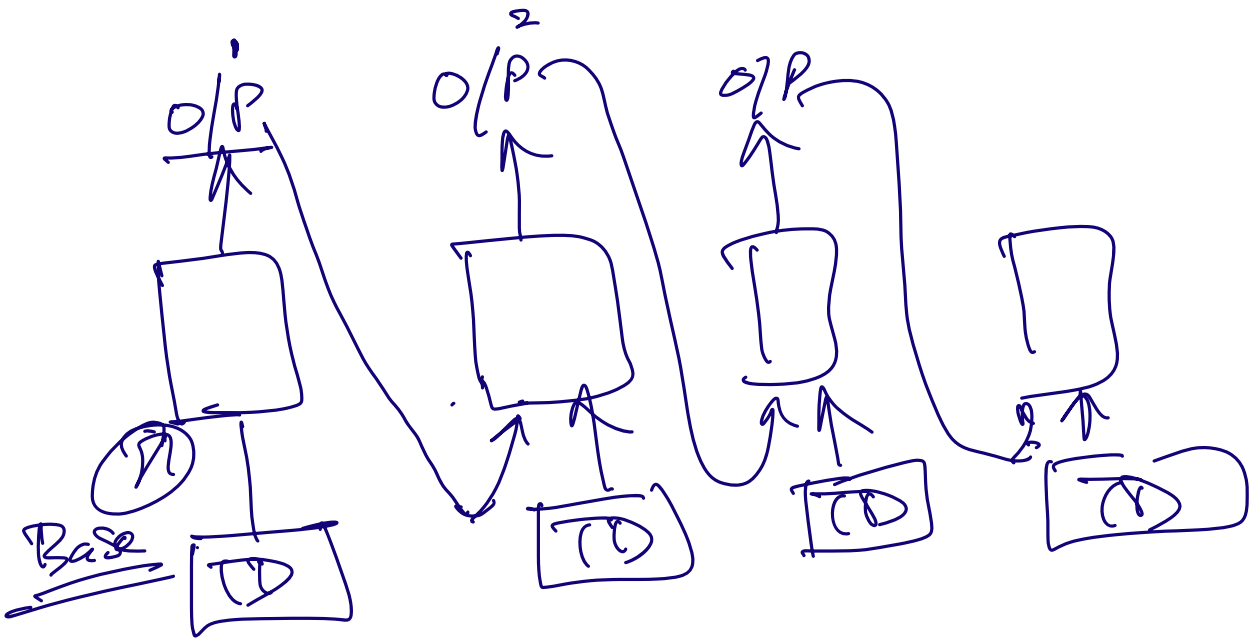
- use random thresholds for randomly chose features
- Trade bias for lower variance
- check this out if Random Forest overfit
- By default in this approach Bootstrapping

## Feature Importance

- By checking how much each feature reduced gini impurity you can check relative importance of features

## Boosting

- Combine many weak predictors to produce a strong predictor
- Predictors train sequentially
- Adaptive Boosting & Gradient Boosting




## Adaptive Boosting

### Weighted error rate of $j^{th}$ predictor

$$r_j = \frac{\sum_{\substack{i=1 \\ \hat{y}_j^{(i)} \neq y^{(i)}}}^{m} w^{(i)}}{}, \quad w \to \frac{1}{m}$$

### Predictor weight

$$\alpha_j = \eta \log \frac{1-r_j}{r_j} + \log(K-1)$$

$\eta$ → learning rate

↑ multiclass → with K classes

### Weight for samples

for $i = 1, 2, \ldots, m$

$$w^{(i)} \begin{cases} w^{(i)} & \text{if } \hat{y}_j^{(i)} = y^{(i)} \\ w^{(i)} \exp(\alpha_j) & \text{if } \hat{y}_j^{(i)} \neq y^{(i)} \end{cases}$$

## How does the sample weight get used?

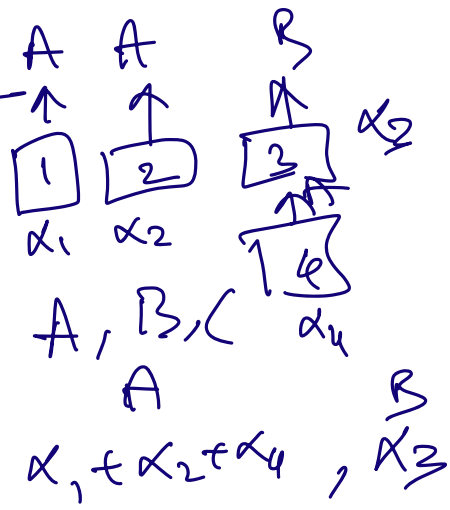
DT → count for the samples will not I but the weight

Logistic Regression → weight will be considered in the cost function

Prediction is done as follows

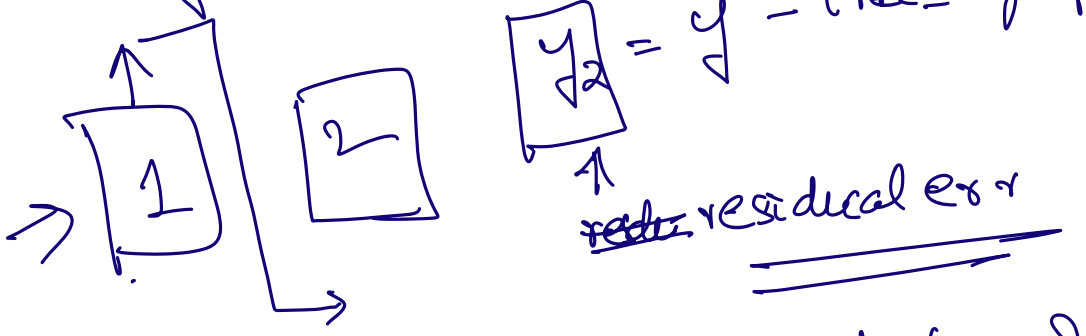
$$\hat{y}(x) = \underset{k}{\operatorname{argmax}} \sum_{j=1}^{N} \alpha_j \, y_j(x) = k$$

k → class




## Gradient Boosting

- sequential predictors
- Each successor tries to fit the residual error from the predecessor




- Learning rate → scales the contribution of each tree

When choose higher learning rate

- you need lesser no. of predictors but high variance
- Can use random sampling of training set in GB